



武汉大学

WUHAN UNIVERSITY

数据结构与程序设计

(Python语言)

DATA STRUCTURE AND PROGRAM DESIGN IN PYTHON

第5讲 数据结构 (二)



Python课程组



2025年02月



CONTENTS

目 录

1 歌曲推荐程序

2 字典

3 字典对象的方法

4 集合

5 集合运算

6 实践指导



武汉大学
WUHAN UNIVERSITY

01

歌曲推荐程序



引例：歌曲推荐程序



武汉大学
WUHAN UNIVERSITY

■ 应用场景

- ✿ 当你想听歌曲但又不知道听什么的时候，你会如何选择？
- ✿ 使用音乐应用程序帮助你推荐曲目。
- ✿ 那么，这些程序如何知道你喜欢听什么类型的歌曲？
- ✿ 又是如何向你推荐歌曲的呢？



引例：歌曲推荐程序



■ 简单的推荐算法：

- ✱ (1) 创建一个数据集，记录一些用户和他们喜欢的歌曲清单；
- ✱ (2) 输入用户A喜欢听的歌曲清单；
- ✱ (3) 查找与用户A最相似的用户 u_x 及其喜欢的歌曲清单，即用户 u_x 喜欢的歌曲与用户A喜欢的歌曲重叠数最多；
- ✱ (4) 推荐用户 u_x 喜欢的歌曲中用户A还没听过的那些歌曲。



引例：歌曲推荐程序



数据集

✱ 例如，已知有7位用户和他们喜欢的歌曲如下：

- 用户u1喜欢的歌曲是：《时光背面的我》、《我想要》、《南半球与北海道》。
- 用户u2喜欢的歌曲是：《你的眼睛像星星》、《百年孤寂》、《执迷不悟》、《一路生花》。
- 用户u3喜欢的歌曲是：《海底》、《时光背面的我》。
- 用户u4喜欢的歌曲是：《你头顶的风》、《时光洪流》、《自愈》、《我们俩》。
- 用户u5喜欢的歌曲是：《南半球与北海道》、《可不可以》、《假面舞会》、《百年孤寂》、《一路生花》。
- 用户u6喜欢的歌曲是：《可不可以》、《时光背面的我》。
- 用户u7喜欢的歌曲是：《执迷不悟》、《不知所措》、《善变》、《一路生花》。



引例：歌曲推荐程序



■ 输入用户A喜欢的歌曲清单

✱ 假设他喜欢的歌曲是：《南半球与北海道》、《一路生花》、《百年孤寂》。

■ 统计用户A与数据集中各候选用户的歌曲重叠数

候选用户	u1	u2	u3	u4	u5	u6	u7
用户A与候选用户喜欢的歌曲重叠数	1	2	0	0	3	0	1



引例：歌曲推荐程序



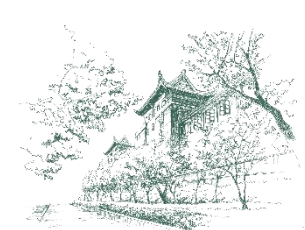
■ 应该用什么类型的对象存储用户及其喜欢的歌曲呢？

■ 设计数据结构

✿ 方法一

```
# 用列表存储用户数据
user_data = [
    ['u1', '时光背面的我', '我想要', '南半球与北海道'],
    ['u2', '你的眼睛像星星', '百年孤寂', '执迷不悟', '一路生花'],
    ['u3', '海底', '时光背面的我'],
    ['u4', '你头顶的风', '时光洪流', '自愈', '我们俩'],
    ['u5', '南半球与北海道', '可不可以', '假面舞会', '百年孤寂', '一路生花'],
    ['u6', '可不可以', '时光背面的我'],
    ['u7', '执迷不悟', '不知所措', '善变', '一路生花']
]

# 查询用户喜欢的歌曲
for i in range(len(user_data)):
    if user_data[i][0]=='u4':
        print("用户喜欢的歌曲是", user_data[i][1:])
```



引例：歌曲推荐程序



设计数据结构

方法二

```
# 用两个列表分别存储一组用户名和他们喜欢的歌曲名称
user_list = ['u1', 'u2', 'u3', 'u4', 'u5', 'u6', 'u7']
song_list = [
    ['时光背面的我', '我想要', '南半球与北海道'],
    ['你的眼睛像星星', '百年孤寂', '执迷不悟', '一路生花'],
    ['海底', '时光背面的我'],
    ['你头顶的风', '时光洪流', '自愈', '我们俩'],
    ['南半球与北海道', '可不可以', '假面舞会', '百年孤寂', '一路生花'],
    ['可不可以', '时光背面的我'],
    ['执迷不悟', '不知所措', '善变', '一路生花']
]

# 查询用户喜欢的歌曲
print("用户u4喜欢的歌曲是：", song_list[user_list.index('u4')])
```



引例：歌曲推荐程序



思考

✿ 这两种数据结构的设计方法，在实现推荐算法时可能存在哪些问题？

更合适的数据结构

- ✿ 保存用户名和他喜欢的歌曲清单，以及关联关系；
- ✿ 根据用户名迅速找到他的歌曲清单。



武汉大学
WUHAN UNIVERSITY

02

字典



字典



字典（类型名：**dict**）是Python中唯一内置的映射类型。

✿ 一个字典对象包含若干“**键-值**”对形式的元素，通过“**键**”来访问“**值**”，类似日常使用新华字典查找一个汉字的解释。

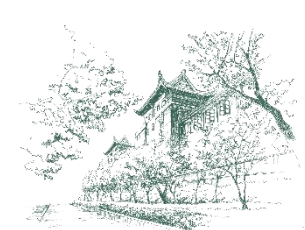
➤ 键（**key**）

➤ 值（**value**）

✿ **键不能重复，只能使用不可变对象**（bool, int, float, complex, str, tuple, frozenset等）。

✿ 可以通过指定的键从字典访问值。

✿ 别称：关联数组、映射、散列表。



字典的字面值

✿ 字面上，字典对象使用一对大括号把一组元素括起来，元素之间用逗号隔开。

✿ 键值对的书写形式为

键：值

✿ 举例

```
# 用字典存储一组水果的价格
```

```
prices = {'苹果':6.8, '香蕉':3.6, '西瓜':4.5, '毛桃':2.7}
```

```
# 创建更多的字典对象
```

```
empty_dict = { } # 创建空字典
```

```
weekdays = {1:'MON',2:'TUE',3:'WED',4:'THU',5:'FRI',6:'SAT',0:'SUN'} # 使用数字作为键
```

```
one_valid_dict = {(90,100):'优秀', 6:[1,2,3]} # 使用元组作为键，列表作为值
```



案例：歌曲推荐程序

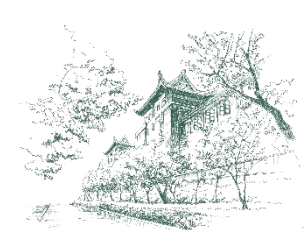


歌曲推荐程序的数据结构设计

方法三：用字典来表示数据集

```
user_db = {  
    'u1': ['时光背面的我', '我想要', '南半球与北海道'],  
    'u2': ['你的眼睛像星星', '百年孤寂', '执迷不悟', '一路生花'],  
    'u3': ['海底', '时光背面的我'],  
    'u4': ['你头顶的风', '时光洪流', '自愈', '我们俩'],  
    'u5': ['南半球与北海道', '可不可以', '假面舞会', '百年孤寂', '一路生花'],  
    'u6': ['可不可以', '时光背面的我'],  
    'u7': ['执迷不悟', '不知所措', '善变', '一路生花']  
}
```

如何根据用户名找到他的歌曲清单？



引用元素

✿ 引用字典元素就是根据键来“查字典”一般形式为：

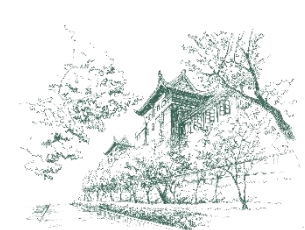
字典[键表达式]

➤ 其中，“字典”是字典对象，形式上可以是字典字面值、引用字典对象的变量；键表达式的值必须是作为字典键的不可变对象。

```
# 在水果价格字典中查香蕉的价格  
print( prices['香蕉'] ) # 输出 3.6
```

✿ 歌曲推荐程序

```
# 查询用户喜欢的歌曲  
print("用户u4喜欢的歌曲是：", user_db['u4'])
```

引用元素

✿ 要注意的是，**不能用索引**来引用字典元素，只能通过**键**来引用元素。

➤ 例如，对于表达式`prices[2]`，Python解释器将把方括号中的2当作一个键，而`prices`中没有2，于是就会报键错误（`KeyError`）。

✿ 为了避免引用元素时出现键错误，一般会先检查键是否存在

```
# 在水果价格字典中查橙子的价格，但橙子并不在字典中
if '橙子' in prices:
    print(prices['橙子']) # 不会执行这条语句
else:
    print('不知道橙子的价格') # 执行后输出 不知道橙子的价格
```

✿ 思考：如何用for语句遍历字典？



■ 用for语句依次访问字典元素

✿ 为了避免引用元素时出现键错误，一般会先检查键是否存在

```
# 遍历prices字典中的键，即水果名称  
for item in prices:  
    print(item)
```

苹果
香蕉
西瓜
毛桃

✿ 思考：若要遍历字典prices，输出每个元素的键和值，应该怎样修改程序？



创建字典对象



■ 用函数dict创建字典对象

✿ 字典的类型名称dict也可以被当作内置函数，用来创建字典对象。

```
another_empty_dict = dict()  # 创建一个空字典

# 创建一个字典保存一位学生的姓名、性别和成绩
student = dict(name='赵元', gender='男', scores=(91, 88, 86))

print(another_empty_dict)
print(student)
```

参数名是一个标识符。

```
{ }
{ 'name': '赵元', 'gender': '男', 'scores': (91, 88, 86) }
```

字典元素的键是一个字符串。

```
monthdays = dict(
    Jan=31, Feb=28, Mar=31, Apr=30, May=31, Jun=30,
    Jul=31, Aug=31, Sep=30, Oct=31, Nov=30, Dec=31 )
```



创建字典对象



■ 用类型函数dict创建字典对象

✿ 字典保存了一组键值对，那么可以准备好一组内部结构相似的序列数据，然后调用函数dict来创建字典对象。

✿ 举例

每个元素都是包含2个数据项的对象，分别对应字典元素的键和值。

```
samples = ((1, 1.2), (2, 1.3), (3, 1.1)) # 用元组对象保存了一组样本
sample_dict = dict(samples) # 基于元组对象创建字典对象
print(sample_dict)

score_list = [['赵元', (91, 88, 92)], ['李涛', [85, 83, 90]], ['陈辉', [78, 84, 80]],
              ['王军', [89, 87, 85]]] # 用列表保存的学生成绩表
score_dict = dict(score_list) # 基于列表创建字典保存学生成绩表
print(score_dict)
```

```
{1: 1.2, 2: 1.3, 3: 1.1}
{'赵元': (91, 88, 92), '李涛': [85, 83, 90], '陈辉': [78, 84, 80], '王军': [89, 87, 85]}
```



增加元素



增加字典元素

- 因为字典元素包含键和值，所以增加新元素就是增加一个键值对，需要提供新元素的键和值。
- 例如，往水果价格字典中增加橙子的价格。

```
prices = {'苹果':6.8, '香蕉':3.6, '西瓜':4.5, '毛桃':2.7}
```

```
prices['橙子'] = 4.3 # 往水果价格字典中增加新元素
```

```
print(prices) # 输出结果: {'苹果': 6.8, '香蕉': 3.6, '西瓜': 4.5, '毛桃': 2.7, '橙子': 4.3}
```

- 执行程序时，首先根据键'橙子'去查字典，如果不存在这个键，则增加新元素，否则就是修改元素的值。



修改元素

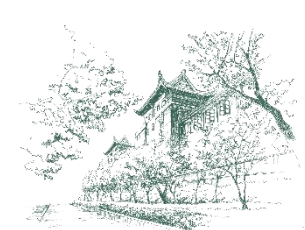


■ 修改字典元素的值

- 字典中元素的键具有唯一性，因此修改字典元素指的就是修改某个元素的值。
- 例如，修改水果价格字典中橙子和西瓜的价格。

```
prices['橙子'] = 3.8 # 修改橙子的价格  
prices['西瓜'] = 4.9 # 修改西瓜的价格  
print(prices) # 输出结果: {'苹果': 6.8, '香蕉': 3.6, '西瓜': 4.9, '毛桃': 2.7, '橙子': 3.8}
```

- 修改元素的值之前也应该先检测元素是否存在，以避免出现键错误。



删除元素



删除字典元素

✿ 可以使用del语句删除字典元素。

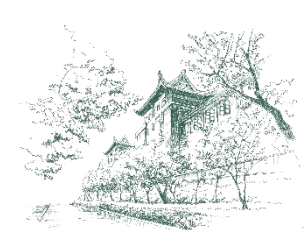
```
del prices['毛桃'] # 从水果价格字典中删除毛桃  
print(prices) # 输出结果: {'苹果': 6.8, '香蕉': 3.6, '西瓜': 4.9, '橙子': 3.8}
```



武汉大学
WUHAN UNIVERSITY

03

字典对象的方法



字典对象的方法



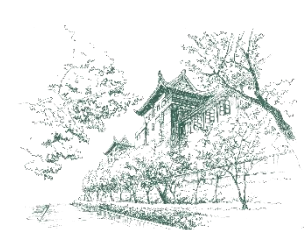
方法keys、values、items

```
all_keys= princs.keys() # 获取全部键  
print(all_keys)
```

```
all_values = princs.values() # 获取全部值  
print(all_values)
```

```
all_items = princs.items() # 获取全部元素  
print(all_items)
```

```
dict_keys(['苹果', '香蕉', '西瓜', '橙子'])  
dict_values([6.8, 3.6, 4.9, 3.8])  
dict_items([('苹果', 6.8), ('香蕉', 3.6), ('西瓜', 4.9), ('橙子', 3.8)])
```



复制字典和清空字典



方法copy

✿ 创建一个新字典对象，新对象的元素与原字典相同。

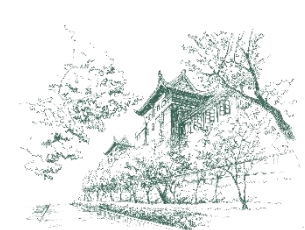
```
new_price_dict = prices.copy() # 把水果价格字典复制一份

print(new_price_dict) #复制的字典: {'苹果': 6.8, '香蕉': 3.6, '西瓜': 4.9, '橙子': 3.8}
print(prices) # 原字典: {'苹果': 6.8, '香蕉': 3.6, '西瓜': 4.9, '橙子': 3.8}
print(new_price_dict is prices) # 复制的字典与原字典是同一个对象吗? False
```

方法clear

✿ 清空字典，即删除所有元素，使得该对象成为空字典。

```
prices.clear()
print(prices) # 输出结果: {}
```

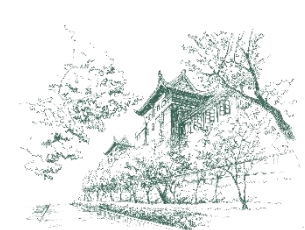



■ 方法update

✿ 用另一个字典（字典B）来更新当前字典。

- （1）对于两个字典中键相同的元素，用字典B中的元素值更新当前字典中的元素值；
- （2）如果字典B中某元素的键在当前字典中不存在，则把该元素添加到当前字典。

```
prices = {'苹果':6.4, '香蕉':3.3, '油桃':4.6}
prices.update(new_price_dict) # 用new_price_table更新prices
print(prices) # 输出结果: {'苹果': 6.8, '香蕉': 3.6, '油桃': 4.6, '西瓜': 4.9, '橙子': 3.8}
print(new_price_dict) # 输出结果: {'苹果': 6.8, '香蕉': 3.6, '西瓜': 4.9, '橙子': 3.8}
```

移除字典元素



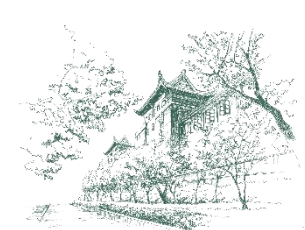
■ 方法pop

✿ 用指定的键查字典

- 如果找到对应元素，则移除该元素，可以赋值给一个变量（引用的是该元素的值）；
- 如果没找到，则报键错误。

```
if '油桃' in prices:
    nectarine = prices.pop('油桃') # 移除油桃，返回其价格
    print(nectarine) # 输出结果: 4.6

print(prices) # 输出结果: {'苹果': 6.8, '香蕉': 3.6, '西瓜': 4.9, '橙子': 3.8}
```



获取字典元素的值



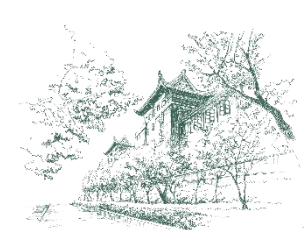
■ 方法get

✱ 根据指定的键查字典

- 如果找到对应元素，则返回该元素的值；
- 否则返回None或者返回指定的值。

```
result = prices.get('地瓜')  
print(result)  # 输出结果: None
```

```
result = prices.get('地瓜', '不知道')  
print(result)  # 输出结果: 不知道
```



字典对象的方法小结



字典对象的方法

方 法	描 述
<code>d.keys()</code>	返回字典d中所有键的列表，类型为dict_keys。
<code>d.values()</code>	返回字典d中值的列表，类型为dict_values。
<code>d.items()</code>	返回字典d中由键和相应值组成的元组的列表，类型为dict_items。
<code>d.clear()</code>	删除字典d的所有条目。
<code>d.copy()</code>	返回字典d的浅复制拷贝，不复制嵌入结构。
<code>d.update(x)</code>	将字典x中的键值加入到字典d。
<code>d.pop(k)</code>	删除键值为k的键值对，返回k所对应的值。
<code>d.get(k[, y])</code>	返回键k对应的值，若未找到该键返回None，若提供y，则未找到k时返回y。



案例：字母频次统计程序



■ 字母频次统计程序

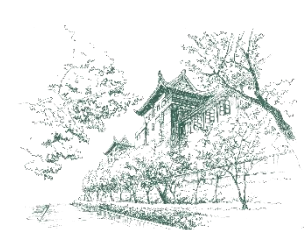
✿ 需求：输入一个字符串，统计其中出现的每个字母字符的频次。

✿ 设计数据结构

- 创建一个字典对象来保存字母字符的频次，每个元素对应一个字符，字符本身作为键，其频次为值。

✿ 算法设计思路

- 读入用户输入的字符串后
- 依次取每一个字符，如果是字母字符，则查频次字典
 - 如果该字符不在字典中，则把该字符加入字典，频次为1；
 - 否则，把字典中该字符的频次增1。



案例：字母频次统计程序



字母频次统计程序

实现

```
text = input("输入一行文本: ")

text = text.lower()  # 把字母字符转换为小写字符

frequencies = {}  # 创建频次字典

for c in text:  # 依次取字符，统计字母字符的频次
    if c.isalpha():
        frequencies[c] = frequencies.get(c, 0) + 1

print("Letter Frequency")  # 输出统计结果
for c,f in frequencies.items():
    print(f"{c:^6} {f:^9}")
```

```
输入一行文本: Beautiful is better than ugly.
Letter Frequency
b      2
e      3
a      2
u      3
t      4
i      2
f      1
l      2
s      1
r      1
h      1
n      1
g      1
y      1
```




武汉大学
WUHAN UNIVERSITY

04

集合



集合



武汉大学
WUHAN UNIVERSITY

■ 集合

- ✱ 包含多个元素，元素不能重复出现，且必须是不可变对象。
- ✱ 支持集合交、并、差等运算。

■ 集合可分为两类

- ✱ 可变集合 (**set**)：可以添加、修改和删除元素。
- ✱ 不可变集合 (**frozenset**)：不允许添加、修改和删除元素，可以用作另一个集合的元素或是字典的键。



■ 集合字面值的一般形式

✿ `{s1, s2, ... sn}`

- 创建的是可变集合 **set**,
- 用逗号分隔的数据项作为集合的一个元素。

```
weekdays = {'MON', 'TUE', 'WED', 'THU', 'FRI', 'SAT', 'SUN'} # 注意元素的顺序
words = {'this', ('is', 'a'), 'mutable', 'set'} # 注意元素的类型和顺序
random_values = {12, 3.4, 5, 67, 8.9, 0.1, 2, 3.4, 5} # 注意元素去重

print(weekdays)
print(words)
print(random_values)
```

```
{'TUE', 'SAT', 'THU', 'MON', 'FRI', 'WED', 'SUN'}
{'mutable', ('is', 'a'), 'set', 'this'}
{0.1, 2, 67, 3.4, 5, 8.9, 12}
```



集合的字面值



■ 书写集合字面值

✿ 注意事项:

(1) 因为“{}”是空字典的字面形式，所以创建空集合，必须调用函数set。

```
empty_set = set()  # 创建空集合，不能写成{ }  
  
print(empty_set)
```

```
set()
```

(2) 集合字面量的元素书写顺序不是集合对象被创建后元素的实际顺序。

(3) 集合字面量可以书写重复元素，但是在集合对象被创建后，重复元素只会保存唯一一个副本。



引用元素和遍历集合

■ 关于集合元素的引用和操作

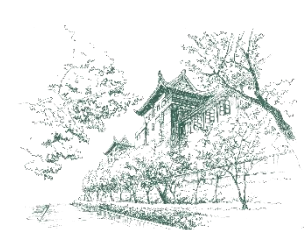
- ✿ 集合中元素的顺序不是固定的，而是会自动调整。
- ✿ 集合不是序列，不能用索引来引用单个集合元素，也不能做切片操作。
- ✿ 集合是可迭代对象，一般使用for语句来遍历集合的元素。

```
# 遍历集合
```

```
for value in random_values:  
    print(value, end=' ')
```

```
# 检测对象是否在集合中
```

```
print('this' in words)    # 输出结果: True  
print('is' not in words)  # 输出结果: True
```



创建集合



■ 用函数set基于一个可迭代对象创建集合对象。

'''基于可迭代对象创建集合对象示例'''

```
import random
```

```
random_numbers = [random.randint(1,20) for i in range(10)]
```

```
print(random_numbers) # 输出结果: [14, 12, 9, 14, 12, 11, 19, 11, 2, 17]
```

```
numbers = set(random_numbers) # 找到列表中出现了哪些数
```

```
print(numbers) # 输出结果: {2, 9, 11, 12, 14, 17, 19}
```

```
text = "Beautiful is better than ugly"
```

```
letters = set(text) # 找到文本中出现了哪些字符
```

```
print(letters) # 输出结果: {'g', 'i', 'b', 'u', 'a', 's', 'h', 't', 'B', 'n', ' ', 'f', 'r', 'y', 'l', 'e'}
```

```
letters = list(letters) # 基于集合创建列表
```

```
letters = sorted(letters) # 对列表中的字符进行排序
```

```
print(letters) # 输出结果: [' ', 'B', 'a', 'b', 'e', 'f', 'g', 'h', 'i', 'l', 'n', 'r', 's', 't', 'u', 'y']
```

去重应用



操作集合对象的方法

集合对象的方法

复制集合 (copy)

增加 (add)、删除 (remove)、更新 (update) 和清空 (clear) 集合元素

```
colors = {'红', '黄', '蓝'}  
print(colors)  # 输出结果: {'红', '蓝', '黄'}
```

```
colors.add('绿')  # 增加新元素  
print(colors)  # 输出结果: {'绿', '红', '蓝', '黄'}
```

```
colors.update({'红', '橙', '黄', '绿', '青', '蓝', '紫'})  # 把另一个集合中的元素并入当前集合  
print(colors)  # 输出结果: {'绿', '青', '橙', '黄', '蓝', '红', '紫'}
```

```
pallete = colors.copy()  # 复制集合, 即基于当前集合创建一个元素相同的新集合  
print(pallete)  # 输出结果: {'绿', '蓝', '红', '青', '橙', '黄', '紫'}
```

```
colors.clear()  # 清空集合  
print(colors)  # 输出结果: set()
```



用内置函数操作集合



■ 通过内置函数len、max、min、sum获取集合的长度、元素最大值、元素最小值、元素之和。

```
'''生成10个1~20内不同的整数，找到最大数和最小数，并计算平均值'''
```

```
import random
```

```
numbers = set()
```

```
while len(numbers)<10:
```

```
    numbers.add(random.randint(1, 20))
```

```
print(numbers)
```

```
print(f"最大数是{max(numbers)}")
```

```
print(f"最小数是{min(numbers)}")
```

```
print(f"平均值是{sum(numbers)/len(numbers)}")
```

```
{2, 3, 6, 7, 8, 10, 12, 13, 18, 20}
```

```
最大数是20
```

```
最小数是2
```

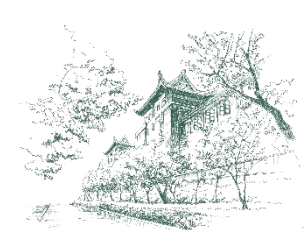
```
平均值是9.9
```



武汉大学
WUHAN UNIVERSITY

05

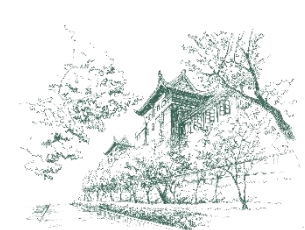
集合运算



集合支持的运算

✱ 并集、交集、差集和异或集运算符：|、&、-、^

运 算	描 述
$s1 \mid s2$	并集，求s1和s2中的所有元素
$s1 \& s2$	交集，求s1和s2中相同元素
$s1 - s2$	差集，求s1中不在s2中的元素
$s1 \wedge s2$	异或集，求s1与s2中相异元素



■ 示例：集合在歌曲推荐程序中的应用

歌曲推荐程序中有两个用户喜欢的歌曲清单分别用集合对象保存

```
user1 = {'时光背面的我', '我想要', '南半球与北海道', '假面舞会'}
```

```
user2 = {'南半球与北海道', '可不可以', '假面舞会', '百年孤寂', '一路生花'}
```

找出两个用户喜欢的所有歌曲

```
all_favorite = user1 | user2 # 计算集合user1和user2的并集
```

```
print(all_favorite) # 输出结果: {'百年孤寂', '我想要', '可不可以', '一路生花', '南半球与北海道', '时光背面的我', '假面舞会'}
```

找出两个用户都喜欢的歌曲

```
common_favorite = user1 & user2 # 计算集合user1和user2的交集
```

```
print(common_favorite) # 输出结果: {'南半球与北海道', '假面舞会'}
```

找出user1喜欢而user2不喜欢的歌曲

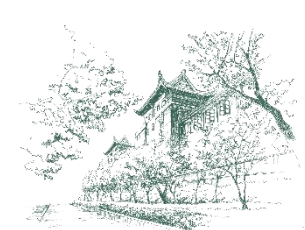
```
only_user1_favorite = user1 - user2 # 计算集合user1和user2的差集
```

```
print(only_user1_favorite) # 输出结果: {'时光背面的我', '我想要'}
```

找出非两个用户共同爱好的歌曲

```
not_common_favorite = user1 ^ user2 # 计算集合user1和user2的异或集
```

```
print(not_common_favorite) # 输出结果: {'百年孤寂', '时光背面的我', '我想要', '可不可以', '一路生花'}
```



集合支持的复合赋值运算



集合支持下面的复合赋值运算

✿ $|=$ 、 $\&=$ 、 $-=$ 、 $\wedge=$

运 算	描 述
$s1 \mid= s2$	并集（与 $s1.update(s2)$ 等效），把 $s2$ 的所有元素加入 $s1$
$s1 \&= s2$	交集（与 $s1.intersection_update(s2)$ 等效），在 $s1$ 中仅保留 $s1$ 和 $s2$ 的相同元素
$s1 -= s2$	差集（与 $s1.difference_update(s2)$ 等效），从 $s1$ 中移除在 $s2$ 中出现的元素
$s1 \wedge= s2$	对称差集（与 $s1.symmetric_difference_update(s2)$ 等效），在 $s1$ 中保留仅出现在 $s1$ 或 $s2$ 中的元素

✿ 要注意的是，这些运算都会直接修改运算符左边的集合，而不是创建新集合。



集合支持的复合赋值运算



■ 示例

```
'''集合支持的复合赋值运算符'''
colors = {'红', '黄', '蓝', '黑', '白'}
rainbow_colors = {'红', '橙', '黄', '绿', '青', '蓝', '紫'}

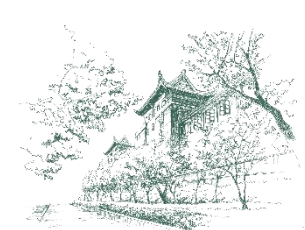
colors |= rainbow_colors # 把集合rainbow_colors中的元素并入左边的集合colors
print(colors) # 输出: {'黄', '红', '黑', '青', '蓝', '绿', '橙', '紫', '白'}

colors -= rainbow_colors # 从集合colors中移除在集合rainbow_colors中出现的元素
print(colors) # 输出: {'黑', '白'}

colors &= rainbow_colors # 在colors中仅保留同时出现在colors和rainbow_colors中的元素
print(colors) # 输出: set()

colors ^= rainbow_colors # 在colors中保留仅出现在colors或rainbow_colors中的元素
print(colors) # 输出: {'黄', '青', '红', '蓝', '绿', '橙', '紫'}
```

✿ 思考：怎么证明一直更新的是colors最初引用的集合？



集合支持的复合赋值运算



■ 示例

'''与集合支持的复合赋值运算符等效的方法'''

```
colors = {'红', '黄', '蓝', '黑', '白'}
```

```
rainbow_colors = {'红', '橙', '黄', '绿', '青', '蓝', '紫'}
```

```
colors.update(rainbow_colors) # 把集合rainbow_colors中的元素并入左边的集合colors
```

```
print(colors) # 输出: {'黄', '红', '黑', '青', '蓝', '绿', '橙', '紫', '白'}
```

```
colors.difference_update(rainbow_colors) # 从集合colors中移除在集合rainbow_colors中出现的元素
```

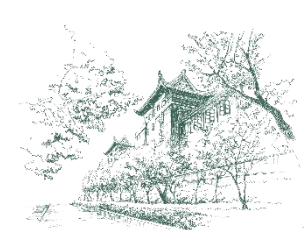
```
print(colors) # 输出: {'黑', '白'}
```

```
colors.intersection_update(rainbow_colors) # 在colors中仅保留同时出现在colors和rainbow_colors中的元素
```

```
print(colors) # 输出: set()
```

```
colors.symmetric_difference_update(rainbow_colors) # 在colors中保留仅出现在colors或rainbow_colors中的元素
```

```
print(colors) # 输出: {'黄', '青', '红', '蓝', '绿', '橙', '紫'}
```



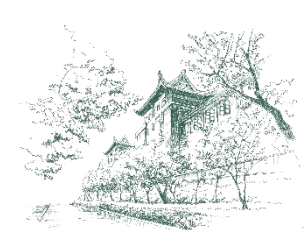
集合的关系运算



集合支持的关系运算

- 关系运算 ($=$ 、 $\neq$ 、 $>$ 、 $\geq$ 、 $<$ 、 $\leq$) 适用于集合对象。
- 集合比较时不必考虑元素的顺序。

运 算	描 述
$s1 \neq s2$	判断集合是否不同
$s1 == s2$	判断集合是否相等
$s1 \leq s2$	判断s1是否是s2的子集
$s1 < s2$	判断s1是否是s2的真子集
$s1 \geq s2$	判断s1是否是s2的超集
$s1 > s2$	判断s1是否是s2的真超集



集合关系运算的应用



集合支持的关系运算

示例

```
pallette = {'蓝', '绿', '橙', '红', '黑', '黄', '紫', '青', '白', '灰'}
rainbow = {'红', '橙', '黄', '绿', '青', '蓝', '紫'}
image1 = {'黑', '白', '灰'}
image2 = {'绿', '蓝', '红'}
image3 = {'红', '黄', '蓝'}
image4 = {'白', '灰', '黑'}

print(image1 == image4) # 输出结果: True
print(image2 != image3) # 输出结果: True
print(pallette > rainbow) # 输出结果: True
print(image1 >= image4) # 输出结果: True
print(image2 < rainbow) # 输出结果: True
print(image1 <= image4) # 输出结果: True
```



集合对象的运算方法



实现集合运算的另一类方法

方 法	描 述
<code>s1.union(s2)</code>	与 $s1 \mid s2$ 等效，返回一个新的集合对象
<code>s1.difference(s2)</code>	与 $s1 - s2$ 等效，返回一个新的集合对象
<code>s1.intersection(s2)</code>	与 $s1 \& s2$ 等效，返回一个新的集合对象
<code>s1.symmetric_difference(s2)</code>	与 $s1 \wedge s2$ 等效，返回一个新的集合对象

注意此表中的方法

- 都不会对 **s1** 做修改，而是返回新的集合对象，被称为非原地方法。
- **s1** 可以是可变集合，也可以是不可变集合。
- **s2** 还可以是其他可迭代对象（如列表、元组）。

课外练习

与修改s1的方法进行比较



武汉大学
WUHAN UNIVERSITY

06

实践指导



应用案例1：歌曲推荐程序



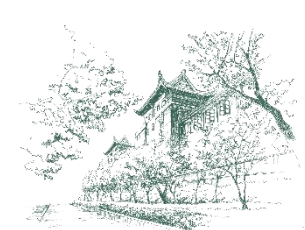
■ 实现歌曲推荐程序

✿ 再次调整数据结构的设计

- 每位用户喜欢的歌曲清单改用集合保存，以便对不同用户歌曲清单进行比较运算。

✿ 设计推荐算法

- 在创建用户数据库后，模拟程序的当前用户，输入他喜欢的歌曲清单，
- 然后，用当前用户的歌曲清单逐一与数据库中每个用户的歌曲清单进行比较，找出共同喜欢的歌曲数量最大的用户（即爱好最相似的用户），
- 最后，把找到的爱好最相似用户的歌曲清单中当前用户还未列出的歌曲作为推荐选项。



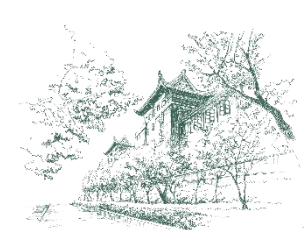
应用案例1：歌曲推荐程序



■ 实现歌曲推荐程序

用字典和集合存储用户数据库

```
user_db = {  
    'u1': {'时光背面的我', '我想要', '南半球与北海道'},  
    'u2': {'你的眼睛像星星', '百年孤寂', '执迷不悟', '一路生花'},  
    'u3': {'海底', '时光背面的我'},  
    'u4': {'你头顶的风', '时光洪流', '自愈', '我们俩'},  
    'u5': {'南半球与北海道', '可不可以', '假面舞会', '百年孤寂', '一路生花'},  
    'u6': {'可不可以', '时光背面的我'},  
    'u7': {'执迷不悟', '不知所措', '善变', '一路生花'}  
}
```

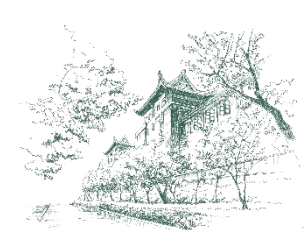


应用案例1：歌曲推荐程序



■ 实现歌曲推荐程序

```
# 当前用户输入几首他喜欢的歌曲，保存到集合
user_songs = set() # 创建空集合对象，准备保存后续输入的歌曲名称
print("输入几首你喜欢的歌曲名称：")
song = input("歌曲名（直接按Enter将结束输入）：")
while song:
    user_songs.add(song) # 把有效的歌曲名添加到集合中
    song = input("歌曲名（直接按Enter将结束输入）：")
```



应用案例1：歌曲推荐程序

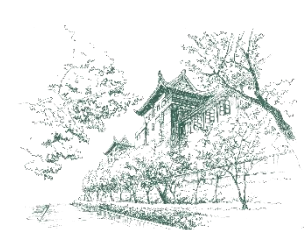


■ 实现歌曲推荐程序

```
# 依次与数据库中的每个用户比较，找到爱好最相似用户
common_songs_count = 0 # 记录当前爱好最相似用户的共同喜欢的歌曲的数量
for u_name in user_db:
    count = len(user_songs & user_db[u_name]) # 共同喜欢的歌曲的数量
    if common_songs_count < count:
        common_songs_count = count # 更新当前找到的爱好最相似用户的共同爱好的歌曲的数量
        similar_user_name = u_name # 更新当前找到的爱好最相似用户的名称

# 输出爱好最相似用户
print(f"与你爱好最相似的用户是{similar_user_name}")
print(f"他喜欢的歌曲清单: {user_db[similar_user_name]}")
print(f"想你推荐他喜欢的歌曲中你还没听过的歌曲: {user_db[similar_user_name] - user_songs}")
```

✿ 思考：此程序还可能存在哪些考虑不周的问题？如何改进？



应用案例2：数据集查询分析

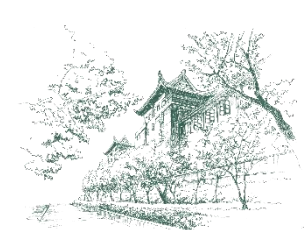


■ 学生选课信息查询

- ✿ 若干学生的选课数据表，每个学生包含学号、姓名、专业、年级、若干选修课程名称。
- ✿ 实现以下查询分析：
 - 有哪些专业的学生选修了《Python》课程，输出学生的学号、姓名、专业和年级
 - 2023级的学生都选修了哪些课程

■ 思考

- ✿ 数据集的数据结构
- ✿ 用哪类对象表示数据集和各项数据



应用案例2：数据集查询分析



武汉大学
WUHAN UNIVERSITY

■ 学生选课信息查询

✱ 数据结构

基于列表、字典和集合表示学生选课数据表

```
stu_db = [  
    {'学号': 20221101, '姓名': '陈东', '专业': '新闻与传播', '年级': 2022,  
     '选修课程': {'Python', '数字人文'}},  
    {'学号': 20231102, '姓名': '赵茜', '专业': '新闻与传播', '年级': 2023,  
     '选修课程': {'数据科学导论', 'Python', '数据分析'}},  
    {'学号': 20233303, '姓名': '王楠', '专业': '建筑学', '年级': 2023,  
     '选修课程': {'数据科学导论', 'Python', '人工智能与机器学习'}},  
    {'学号': 20225104, '姓名': '胡蓓', '专业': '历史', '年级': 2022,  
     '选修课程': {'Python', '数字人文'}},  
    {'学号': 20226105, '姓名': '袁钟', '专业': '哲学', '年级': 2022,  
     '选修课程': {'数据科学导论', '数字人文'}}  
]
```


应用案例2：数据集查询分析



武汉大学
WUHAN UNIVERSITY

学生选课信息查询

数据查询

```
# 有哪些专业的学生选修了《Python》课程
print("以下学生选修了《Python》课程")
for student in stu_db:
    if 'Python' in student['选修课程']:
        print(student['学号'], student['姓名'], student['专业'], student['年级'])

print()
# 2023级学生都选修了哪些课程
courses = set() # 创建空集合, 准备保存找到的课程名称
for student in stu_db:
    if student['年级']==2023:
        courses |= student['选修课程']
print("2023级的学生选修了这些课程: ", courses)
```

以下学生选修了《Python》课程
20221101 陈东 新闻与传播 2022
20231102 赵茜 新闻与传播 2023
20233303 王楠 建筑学 2023
20225104 胡蓓 历史 2022

2023级的学生选修了这些课程: {'Python', '数据分析', '人工智能与机器学习', '数据科学导论'}

尝试更多练习

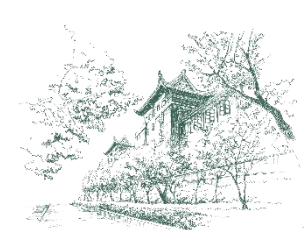


标准库的collections模块

collections模块简介

- ✿ 标准库中的模块。
- ✿ 实现了一些专门化的容器，是对Python的常用内置容器（列表、元组、字典和集合等）的补充。

类 名	描 述
deque	类似列表的容器，但 append 和 pop 在其两端的速度更快。
ChainMap	类似字典的类，用于创建包含多个映射的单个视图。
Counter	用于计数可迭代对象的字典子类
OrderedDict	字典的子类，能记住条目被添加进去的顺序。
defaultdict	字典的子类，通过调用用户指定的工厂函数，为键提供默认值。



应用案例3：字母频次统计程序（改版）



武汉大学
WUHAN UNIVERSITY

■ 用Counter类实现字母频次统计

✱ Counter类可以更加快速地实现频次统计，并且能够提供更多的功能。

✱ 用Counter类来改写字母字符频次统计程序

```
from collections import Counter

text = input("输入一行文本: ")
text = text.lower()  # 把字母字符转换为小写字母

for c in ' .,':  # 剔除掉文本中的空格和标点符号
    text = text.replace(c, '')

# 调用Counter函数创建计数对象，统计文本中出现的每个字符的频次，保存在字典中
frequencies = Counter(text)

print("Letter Frequency")  # 输出频次最高的3个字符
for c, f in frequencies.most_common(3):
    print(f"{c:^6} {f:^9}")
```

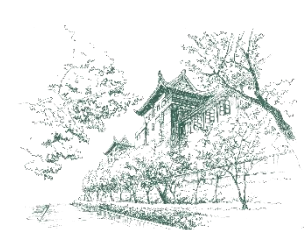
```
输入一行文本: Beautiful is better than ugly.
Letter Frequency
t         4
e         3
u         3
```



■ Matplotlib简介

- ✿ Python扩展库，用于创建静态、动态和交互式可视化效果。
- ✿ <https://matplotlib.org/>
- ✿ Matplotlib包含多个模块，每个模块都提供了特定的功能，从基本的绘图到高级的自定义绘图和动画等。
- ✿ matplotlib.pyplot是最常用的模块，它提供了一个命令式绘图接口，用于快速生成图形。
 - 通常使用下面的语句导入matplotlib.pyplot模块：

```
import matplotlib.pyplot as plt
```



用Matplotlib绘图



武汉大学
WUHAN UNIVERSITY

■ 用Matplotlib绘图的步骤

✿ (1) 创建画布与子图

➤ 方法一

```
import matplotlib.pyplot as plt # 导入matplotlib.pyplot模块（需确保已经安装了Matplotlib库）

# 创建画布，默认尺寸为640x480，单位像素
fig = plt.figure()

# 添加1行2列，共2个子图，返回第1个子图对象
ax = plt.subplot(1, 2, 1)
```

➤ 方法二

```
import matplotlib.pyplot as plt

# 创建画布，包含1行2列，共2个子图，返回画布对象和子图数组对象
fig, axs = plt.subplots(1, 2)
```

➤ 如果只是绘制一幅简单的图形，那么创建画布和子图的步骤可以省略，绘图函数将自动创建画布和1个子图。



■ 用Matplotlib绘图的步骤

✿ (2) 添加绘图元素

➤ 下表列举了常用绘图元素及对应函数。

类 别	绘图元素	函数调用示例	子图方法调用示例
基础图形绘制	折线图	<code>plt.plot(x, y)</code>	<code>ax.plot(x, y)</code>
	散点图	<code>plt.scatter(x, y)</code>	<code>ax.scatter(x, y)</code>
	柱状图（纵向）	<code>plt.bar(x, height)</code>	<code>ax.bar(x, height)</code>
	柱状图（横向）	<code>plt.barh(y, width)</code>	<code>ax.barh(y, width)</code>
	直方图	<code>plt.hist(data, bins)</code>	<code>ax.hist(data, bins)</code>
	饼图	<code>plt.pie(sizes, labels)</code>	<code>ax.pie(sizes, labels)</code>
	箱线图	<code>plt.boxplot(data)</code>	<code>ax.boxplot(data)</code>
	面积图	<code>plt.fill_between(x, y1)</code>	<code>ax.fill_between(x, y1)</code>
	热力图	<code>plt.imshow(data)</code>	<code>ax.imshow(data)</code>
	等高线图	<code>plt.contour(X,Y,Z)</code>	<code>ax.contour(X,Y,Z)</code>
	极坐标图	<code>plt.polar(theta, r)</code>	<code>ax.polar(theta, r)</code>



■ 用Matplotlib绘图的步骤

✿ (2) 添加绘图元素

➤ 下表列举了常用绘图元素及对应函数。

标题与标签添加	标题	<code>plt.title(text)</code>	<code>ax.set_title(text)</code>
	轴标签	<code>plt.xlabel(text)</code> <code>plt.ylabel(text)</code>	<code>ax.set_xlabel(text)</code> <code>ax.set_ylabel(text)</code>
坐标轴设置	坐标轴刻度	<code>plt.xticks(ticks, labels)</code> <code>plt.yticks(ticks, labels)</code>	<code>ax.set_xticklabels(labels)</code> <code>ax.set_yticklabels(labels)</code>
	坐标轴范围	<code>plt.xlim(min, max)</code> <code>plt.ylim(min, max)</code>	<code>ax.set_xlim(min, max)</code> <code>ax.set_ylim(min, max)</code>
辅助元素添加	图例	<code>plt.legend(labels)</code>	<code>ax.legend(labels)</code>
	文本	<code>plt.text(x, y, text)</code>	<code>ax.text(x, y, text)</code>
	网格线	<code>plt.grid(True)</code>	<code>ax.grid(True)</code>
其他	调整布局	<code>plt.tight_layout()</code> <code>plt.subplots_adjust()</code>	



用Matplotlib绘图



武汉大学
WUHAN UNIVERSITY

■ 用Matplotlib绘图的步骤

✱ (3) 显示和保存画布

➤ 显示画布的函数为：`plt.show()`，保存画布的函数为：`plt.savefig()`。



用Matplotlib绘图



■ 示例：折线图

```
import matplotlib.pyplot as plt
import numpy as np

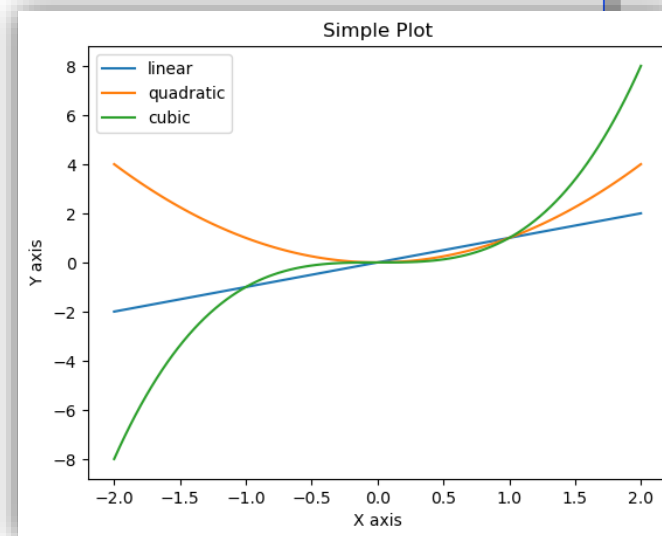
# 生成数据
x = np.linspace(-2, 2, 100)

# 绘制图形
plt.plot(x, x, label='linear') # 绘制直线:  $y = x$ , 图例标签为"linear"
plt.plot(x, x ** 2, label='quadratic') # 绘制抛物线:  $y = x^2$ , 图例标签"quadratic"
plt.plot(x, x ** 3, label='cubic') # 绘制三次曲线:  $y = x^3$ , 图例标签"cubic"

# 添加标题和轴标签
plt.title("Simple Plot") # 添加标题
plt.xlabel('X axis') # 设置x轴标签
plt.ylabel('Y axis') # 设置y轴标签

# 显示图例
plt.legend() # 根据label参数自动生成图例

# 显示图形
plt.show()
```





用Matplotlib绘图



武汉大学
WUHAN UNIVERSITY

■ 示例：包含多个子图的画布

```
import matplotlib.pyplot as plt
import numpy as np

# 创建画布，包含2行2列，共4个子图，参数figsize=(10, 8)设置画布尺寸为10英寸宽、8英寸高
fig, axs = plt.subplots(2, 2, figsize=(10, 8))

# 在子图1（左上）中绘制折线图，图例标签为"Line 1"
axs[0, 0].plot([1, 2, 3, 4], [10, 20, 23, 30], label="Line 1")
axs[0, 0].set_title("Line Plot")
axs[0, 0].set_xlabel("X-axis")
axs[0, 0].set_ylabel("Y-axis")
axs[0, 0].legend()

# 在子图2（右上）中绘制柱状图
categories = ['A', 'B', 'C', 'D']
values = [10, 17, 7, 14]
# 绘制柱状图，参数color='skyblue'定义柱体颜色（支持颜色名称或十六进制码）
axs[0, 1].bar(categories, values, color='skyblue')
axs[0, 1].set_title("Bar Chart")
axs[0, 1].set_xlabel("Categories")
axs[0, 1].set_ylabel("Values")
```

axs是一个二维NumPy数组，数组元素为子图对象，所以子图索引为axs[row, col]。



用Matplotlib绘图



武汉大学
WUHAN UNIVERSITY

■ 示例：包含多个子图的画布

```
# 在子图3（左下）中绘制散点图
x = np.random.rand(80)
y = np.random.rand(80)
# 绘制散点图，参数marker='o'指定散点形状为圆形（默认即为o，可改为^、s等）
axs[1, 0].scatter(x, y, color='red', marker='o')
axs[1, 0].set_title("Scatter Plot")
axs[1, 0].set_xlabel("X-axis")
axs[1, 0].set_ylabel("Y-axis")

# 在子图4（右下）中绘制直方图
data = np.random.randn(1000)
# 绘制直方图，参数bins=30将数据范围分成30个区间统计频次，alpha=0.7设置透明度为70%（避免遮挡网格线）
axs[1, 1].hist(data, bins=30, color='green', alpha=0.7)
axs[1, 1].set_title("Histogram")
axs[1, 1].set_xlabel("Value")
axs[1, 1].set_ylabel("Frequency")

# 调整布局（自动调整子图间距，避免重叠）
plt.tight_layout()

# 显示图形
plt.show()
```

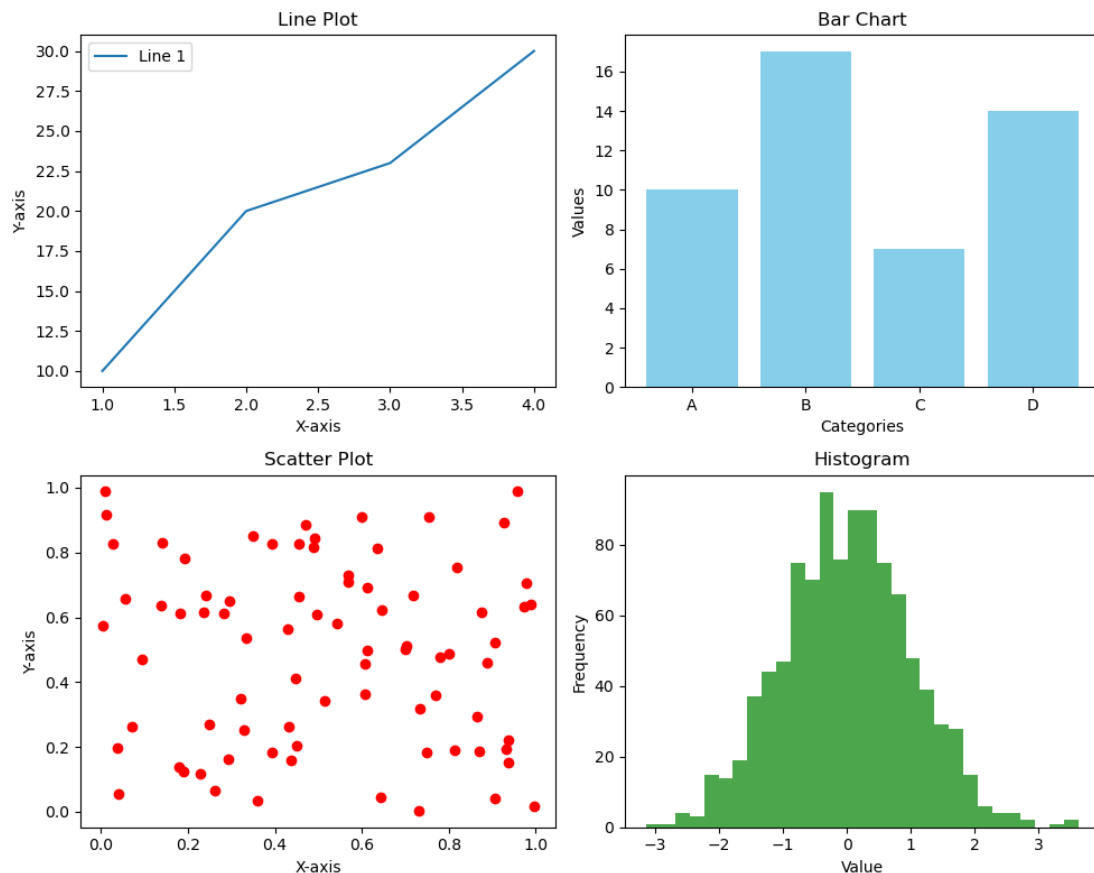


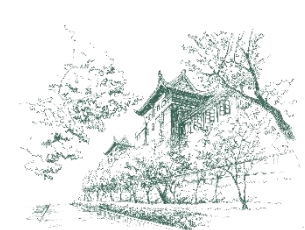
用Matplotlib绘图



武汉大学
WUHAN UNIVERSITY

■ 示例：包含多个子图的画布





应用案例4：字母频次统计结果可视化



武汉大学
WUHAN UNIVERSITY

■ 用柱状图呈现字母频次统计结果

```
from collections import Counter
import matplotlib.pyplot as plt

text = input("输入一行文本: ")

# 文本预处理
text = text.lower()
for c in ' .,':
    text = text.replace(c, '')

# 字母频次统计
frequencies = Counter(text)

# 准备绘图数据（按字母表顺序排序）
sorted_letters = sorted(frequencies.keys())
counts = [frequencies[char] for char in sorted_letters]
```

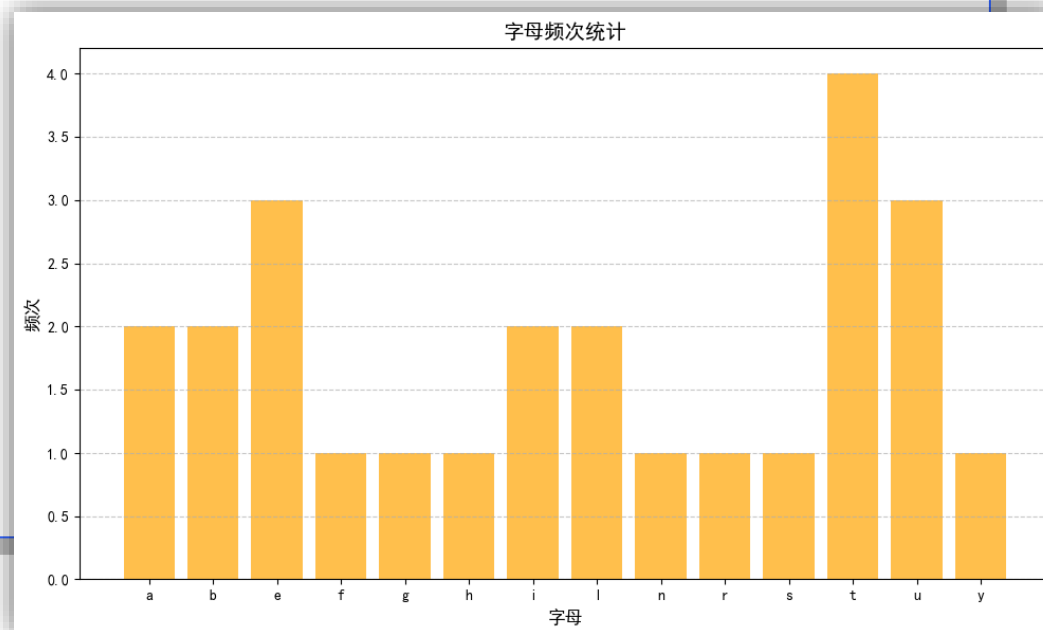


应用案例4：字母频次统计结果可视化



■ 示例：用柱状图呈现字母频次统计结果

```
# 创建画布
plt.figure(figsize=(10, 6))
# 绘制柱状图，柱体颜色为orange，透明度为0.7
plt.bar(sorted_letters, counts, color='orange', alpha=0.7)
# 设置中文字体（黑体），避免中文无法显示
plt.rcParams['font.sans-serif'] = ['SimHei']
# 设置标题，标题字体大小为14，加粗
plt.title('字母频次统计', fontsize=14, fontweight='bold')
# 设置轴标签，标签字体大小为12
plt.xlabel('字母', fontsize=12)
plt.ylabel('频次', fontsize=12)
# 设置网格线，网格线类型为虚线，透明度为0.7
plt.grid(True, axis='y', linestyle='--', alpha=0.7)
# 调整布局
plt.tight_layout()
# 显示图形
plt.show()
```





小结



- 字典适合表示结构复杂的数据。
- 字典的键不允许重复，只能是不可变对象。
- 通过键来快速访问字典元素。
- 集合中没有重复元素。
- 集合运算是集合的突出优势。
- 综合序列、字典和集合，合理设计数据结构，可以降低算法复杂度。
- 标准库的collections模块提供了更多的容器类，实现多种数据结构。
- 扩展库Matplotlib常用来实现数据可视化。



武汉大学

WUHAN UNIVERSITY

谢谢

THANK YOU

